ikerlan

MEMBER OF BASQUE RESEARCH & TECHNOLOGY ALLIANCE



User Guide – SDK Fw.

Status of the procedure:

Author	Reviewed by	Approved by
Hebert Luyo	Ane Mijangos	Ane Mijangos
09/02/2026	12/02/2026	17/02/2026

INDEX

1. Ikerlan 3
2. SDK Overview 4
2.1. Software Architecture overview 4
2.2. Supported products 5
3. SDK Layers 5
3.1. Application Layer 5
3.2. Middleware Layer 5
3.3. Services Layer 6
3.4. Portability Layer 6
3.5. Client Platform Libraries 6
4. SDK Package description 7
5. SDK Public API 7
5.1. API Usage Model 7
5.2. Initialize Function 7
5.3. Authentication Functions 8
5.3.1. Token-to-Server Authentication 8
5.3.2. Server-to-Token Authentication 9
6. Configuration 10
6.1. Enabling Debug Output 10
6.2. Retry Configuration 11
6.3. MQTT Broker Configuration 11
6.4. USB/SPI Port Configuration 11
6.5. CRC Validation 12
7. Proof of Concept (PoC) 13
7.1. PoC Scope and Purpose 13
7.2. Project Structure 13
7.3. Target Environment 14
7.4. PoC Dependencies 14
7.4.1. GCC and Build Tools 15
7.4.2. MQTT Dependencies (Mosquitto) 15
7.5. Building the PoC 15
7.6. Running the PoC 15
7.6.1. Option 1: Normal Flow (No Faults) 16

7.6.2. Option 2: COMM TX Error 18
7.6.3. Option 3: COMM RX Error 19
7.6.4. Option 4: SPI TX Error 20
7.6.5. Option 5: SPI RX Error 21
7.7. Debugging the PoC 22
7.7.1. Debug Setup (Linux Only) 22
7.7.2. Starting a Debug Session 22
7.7.3. Setting Breakpoints 22
7.8. MQTT Broker Configuration 23
7.9. Notes and Limitations 23

1. IKERLAN

IKERLAN is a leading knowledge transfer technological centre providing competitive value to companies. We seek for excellence in R&D&i, thanks to the continuous adaptation to the needs of our customers and the proximity with the business reality. Faithful to our mission, we have been working daily since 1974 to develop solutions that allow our customers to become more and more competitive. We are a cooperative member of the MONDRAGON Corporation and the Basque Research and Technology Alliance (BRTA).

Thanks to a unique cooperation model, which combines technology transfer activities, internal research and training of highly qualified personnel, IKERLAN is currently the trusted technological partner of major companies in the country. To meet our goal, we are structured in two technological specialisation units:

- ELECTRONICS, INFORMATION AND COMMUNICATION TECHNOLOGIES
- ENERGY AND MECHATRONICS

IKERLAN is a centre that is dynamic and open to the world. We are an accredited agent of the Basque Network of Science, Technology and Innovation and of the Spanish Federation of Technology Centres. We have a major cooperation network integrated by renowned European centres and universities, with which we conduct activities of research and training of researchers.

2. SDK OVERVIEW

2.1. Software Architecture overview

The SDK is designed following a **layered architecture** approach. This design separates responsibilities, protects the public API, and ensures full portability across platforms such as PC and embedded microcontrollers (MCUs).

Each layer has a **clear and well-defined role**, allowing the SDK to evolve internally without impacting user applications.

The SDK is divided into the following layers: Application, Middleware, Services and Portability.

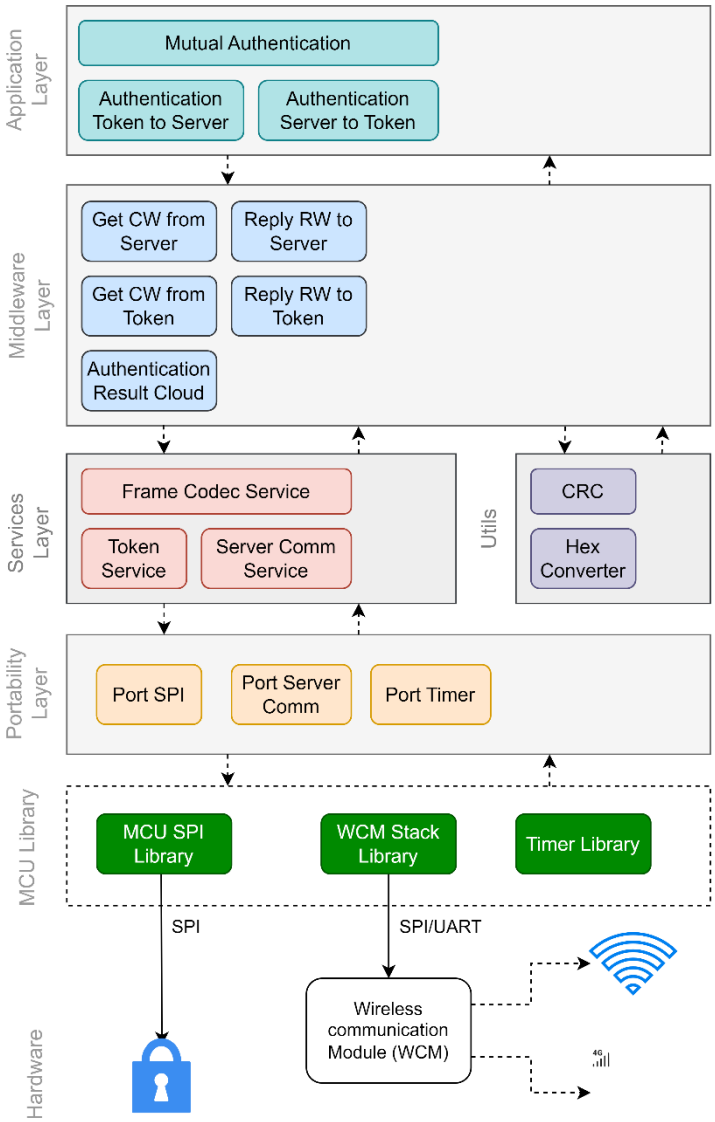


Figure 1. SDK Layered Architecture.

2.2. Supported products

The SDK runs on the following set-up:

Set-up	Port_SPI	Port_COMM
virtualized environment	USB Sandgrain	PC MQTT
MCU 1		
MCU 2		

3. SDK LAYERS

3.1. Application Layer

The Application Layer defines the highest-level behavior of the SDK and represents the functional goal of the system.

In the context of this SDK, the Application Layer is responsible for mutual authentication between a token and a server, including:

- Authentication from Token to Server
- Authentication from Server to Token

This layer expresses what the SDK does from a business and functional perspective. It orchestrates complete authentication flows and exposes functionality through the public SDK API.

The Application Layer does not implement communication protocols, cryptographic operations, or hardware access. All execution details are delegated to lower layers.

3.2. Middleware Layer

The Middleware Layer implements the authentication protocol logic required to achieve mutual authentication.

This layer executes the ordered sequence of operations involved in the authentication process, such as:

- **MW_Get_CW_From_Server:** This function is responsible for obtaining the Challenge Word (CW) from the server and storing it in the device.
- **MW_Reply_RW_To_Server:** This function sends the Response Word (RW) to the server and waits for a response. The response can be either AUTH_OK or AUTH_NOT_OK.
- **MW_Get_CW_From_Token:** This function obtains or generates the Challenge Word (CW) from the token. The CW is generated from the previously computed Response Word (RW), which is 16 bits long. Since the CW must be 32 bits, the RW value is duplicated to build a 32-bit value.
- **MW_Reply_RW_To_Token:** This function obtains the Response Word (RW) from the server using the previously generated Challenge Word (CW). It then calculates the corresponding CW from the token and compares both values to determine whether they match.

- **MW_Authentication_Result_Cloud:** This function sends the authentication result to the cloud. The result can be either AUTH_OK or AUTH_NOT_OK.

The Middleware Layer coordinates interactions between the server and the token, ensures correct sequencing, and enforces protocol rules.

It is fully platform-independent and contains no hardware-specific code.

3.3. Services Layer

The **Services Layer** provides the functional building blocks used by the Middleware Layer to perform concrete actions.

Typical services include:

- Frame Codec Service for message framing, encoding, and decoding
- Token Service for secure token interaction
- Server Communication Service for server-side communication
- Utility services such as CRC calculation and hexadecimal conversion

Each service is focused on a specific responsibility and can be reused across different authentication flows.

3.4. Portability Layer

The **Portability Layer** abstracts platform-dependent capabilities required by the SDK.

This layer provides unified interfaces for:

- SPI communication
- Server communication interfaces
- Timing and delay mechanisms

By isolating platform-specific implementations, the Portability Layer allows the same SDK codebase to run on different platforms without modification.

Porting the SDK to a new platform requires changes only in this layer.

3.5. Client Platform Libraries

The SDK relies on **platform-specific libraries provided by the client** or by the target environment.

These libraries are **not part of the SDK architecture** and are not considered SDK layers.

Typical examples include:

- MCU SPI drivers

- Wireless communication or modem stacks
- Timer or OS abstraction libraries

These libraries are used **exclusively by the Portability Layer** to access platform capabilities. No other SDK layer depends on or interacts with client platform libraries directly.

4. SDK PACKAGE DESCRIPTION

This chapter describes the **package structure of the SGR SDK** and explains how the source code is organized according to the layered architecture.

The SDK package is divided into:

- docs
- examples
- include → Public SDK headers
- port → Port layer interface headers
- src → SDK source code

5. SDK PUBLIC API

This chapter describes the **public application programming interface (API)** exposed by the SGR SDK.

The SDK API provides **high-level functions** that allow the client application to perform mutual authentication operations without requiring knowledge of internal protocols, cryptographic mechanisms, or hardware access.

All SDK API functions are **synchronous** and maintain internal state within the SDK.

5.1. API Usage Model

The SDK follows a **stateful usage model**.

Typical usage flow:

1. Initialize the SDK
2. Perform authentication flows
3. Close the SDK and release resources

The client application must call the API functions **in the correct order**.

5.2. Initialize Function

sgr_sdk_status_t SGR_SDK_Initialize(void)

- **Description**

  Initializes the SGR SDK internal components and prepares the system for authentication operations. This function must be called **once** before invoking any other SDK API.

- **Prototype**

  sgr_sdk_status_t SGR_SDK_Initialize(void);

- **Parameters:** None
- **Return values:**

Value	Description
SGR_SDK_OK	Initialization completed successfully
SGR_SDK_INTERNAL_ERROR	Internal initialization error

- **Usage notes**
  - Must be called before any authentication function.
  - Allocates and initializes internal SDK resources.

5.3. Authentication Functions

5.3.1. Token-to-Server Authentication

- **Description**

  Performs **token-to-server authentication**. This function executes the complete authentication flow, including:

  - Requesting a Challenge Word (CW) from the server
  - Computing the Response Word (RW) using the secure token
  - Sending the Response Word back to the server
  - Processing the authentication result

  All steps are handled internally by the SDK.

- **Prototype:**
  sgr_sdk_status_t SGR_SDK_Authentication_Token_To_Server(void)

- **Parameters:** None.
- **Return values:**

Value	Description
SGR_SDK_OK	Authentication successful
SGR_SDK_AUTH_FAILED	Authentication failed
SGR_SDK_COMM_ERROR	Communication error
SGR_SDK_INTERNAL_ERROR	Internal SDK error

- **Usage notes**
  - Requires the SDK to be initialized.
  - The function blocks until the authentication flow completes.
  - Retry mechanisms are handled internally by the SDK.

5.3.2. Server-to-Token Authentication

- **Description**

  Performs **server-to-token authentication**.

  This function executes the reverse authentication flow, including:

  - Generating a Challenge Word from the token
  - Requesting and validating the Response Word
  - Reporting the authentication result

- **Prototype**

  sgr_sdk_status_t SGR_SDK_Authentication_Server_To_Token(void)

- **Parameters:** None.
- **Return values:**

Value	Description
SGR_SDK_OK	Authentication successful
SGR_SDK_AUTH_FAILED	Authentication failed
SGR_SDK_INTERNAL_ERROR	Internal SDK error

- **Usage notes**
  - Requires the SDK to be initialized.
  - Intended for mutual authentication scenarios.
  - Hardware and protocol details are fully abstracted.

5.6 Error Handling

All SDK API functions return a value of type sgr_sdk_status_t.

The client application must always check the return value to determine the result of the operation.

6. CONFIGURATION

The SGR SDK provides a centralized configuration file, ***sgr_config.h***, which allows the client application to control debugging behavior without modifying any internal SDK source files. All debug-related parameters are grouped in this single file and take effect at compile time.

6.1. Enabling Debug Output

The SDK exposes two independent debug flags, one per module:

MW_DEBUG controls the debug output of the middleware layer. ***PORT_COMM_DEBUG*** controls the debug output of the communication layer, including MQTT publish and receive operations.

To enable debug logging for a module, set its corresponding flag to ***1***. To disable it, either set it to ***0*** or comment out the define entirely:

```
#define MW_DEBUG 1 /* enable middleware debug logs */
#define PORT_COMM_DEBUG 1 /* enable communication debug logs */
```

When a flag is set to ***0*** or undefined, all log calls for that module are replaced by empty statements at compile time, producing no code and no runtime overhead. This makes it safe to ship production builds with debug flags disabled, as there is no performance penalty.

When both flags are enabled, the output will show the full communication flow for each step of the authentication process, including MQTT connection status, the JSON payloads transmitted and received, and the result of each operation. For example:

```
[PORT][MQTT] Connected
[PORT][MQTT] TX about to write | topic=sandgrain/v1/gw-001/req/requestCW | size=118 | data={...}
[PORT][MQTT] Publish OK
[PORT][MQTT] RX received (228 bytes) | data={...}
[MW] ✓ CW received successfully
```

The ***[MW]*** prefix corresponds to **MW_DEBUG** and the ***[PORT][MQTT]*** prefix corresponds to ***PORT_COMM_DEBUG***, making it easy to identify which module generated each log line. Note that the MQTT transport layer is part of the Proof of Concept (PoC) implementation and is not a core component of the SDK itself. In a production integration, the communication layer would be replaced by the target platform's own transport mechanism, and the ***[PORT][MQTT]*** logs would reflect that implementation instead.

6.2. Retry Configuration

The ***MAX_RETRIES*** parameter defines how many times the SDK will retry a failed operation before reporting an error. A value of ***0*** means no retries will be performed and only one attempt will be made. If the client sets a negative value, the SDK will automatically override it and enforce the minimum of ***0***.

```
#if MAX_RETRIES < 0
        #warning "MAX_RETRIES is less than 0, forcing to 0"
        #undef MAX_RETRIES
        #define MAX_RETRIES 0
#endif
```

This validation happens at compile time and will emit a compiler warning to notify the developer that the value was corrected, so the behavior is never silent.

Important: The parameter validation block at the bottom of ***sgr_config.h*** must not be modified. It exists to guarantee safe minimum values are always enforced regardless of what the client configures above it.

6.3. MQTT Broker Configuration

The sgr_config.h file also exposes the MQTT connection parameters that the client must configure to match their own broker setup.

```
#define MQTT_BROKER "172.16.124.248" /* Broker IP or hostname */
#define MQTT_PORT 1883 /* Broker port */
#define MQTT_USER "admin" /* Broker username */
#define MQTT_PASS "changeme" /* Broker password */
#define MQTT_KEEPALIVE 60 /* Keepalive in seconds */
```

The client must replace these values with those corresponding to their own infrastructure. MQTT_BROKER accepts either an IP address or a hostname. MQTT_PORT is typically 1883 for unencrypted connections. MQTT_KEEPALIVE defines the interval in seconds at which the client sends a ping to the broker to keep the connection alive.

It is important to note that the MQTT topic structure used internally by the SDK is built dynamically at runtime using the Device ID read directly from the token chip. The client does not need to configure or hardcode any topic strings, as these are constructed automatically by the communication layer.

6.4. USB/SPI Port Configuration

The SDK communicates with the SGR token via USB/SPI. The port configuration must be set according to your operating system and the specific USB port where the token is connected.

- **Configuration Parameters**

In ***sdk_sgr_config.h***, modify the following parameters based on your platform:

Windows:

```
#define USB_COM_PORT 18 // Change to your COM port number
```

Linux:

```
#define USB_TTY_PORT 0    // Change to your ttyACM number
```

- **Identifying the Correct Port**

Windows

1. Connect the SGR token to your PC
2. Open **Device Manager** (Win + X → Device Manager)
3. Expand **Ports (COM & LPT)**
4. Locate the USB device (typically "USB Serial Port" or "FTDI USB Serial Port")
5. Note the COM port number (e.g., COM3, COM18)
6. Update USB_COM_PORT in sdk_sgr_config.h:

```
#define USB_COM_PORT 3 // If Device Manager shows COM3
```

Linux

1. Connect the SGR token to your PC
2. List available USB serial ports:

   ```
   ls /dev/ttyACM*
   ```

3. Identify your device (usually /dev/ttyACM0)
4. Update USB_TTY_PORT in sdk_sgr_config.h:

```
#define USB_TTY_PORT 0 // For /dev/ttyACM0
```

Note: The SDK automatically converts these values to the correct port string format. You only need to change the numeric value. Some USB devices appear as ttyACM (CDC ACM class) instead of ttyUSB (FTDI/serial converters).

6.5. CRC Validation

The SDK implements CRC16 validation for all received messages to ensure data integrity. This feature can be disabled for testing or debugging purposes.

Configuration

In ***sdk_sgr_config.h***:

```
#define CRC_ENABLE 1 // 1 = Enable, 0 = Disable
```

7. PROOF OF CONCEPT (POC)

This chapter describes the requirements and steps needed to build and execute the Proof of Concept (PoC) provided with the SGR SDK.

The PoC is intended to demonstrate the correct integration and behavior of the SDK in a controlled environment. It is not a mandatory component of the SDK, but a reference implementation used for validation, testing, and demonstration purposes.

7.1. PoC Scope and Purpose

The Proof of Concept validates the following aspects of the SDK:

- Correct initialization and shutdown of the SDK
- Execution of the authentication flows using the public SDK API
- Proper interaction between SDK layers
- Communication with a server using MQTT in a virtualized environment

The PoC runs on a Linux-based system and emulates server-side interactions using an MQTT broker.

7.2. Project Structure

The PoC is structured as a separate application that uses the SDK as a library:

PoC_SGR/
- **.vscode/** → VSCode debug configuration
  - launch.json → Debug settings (Linux + Windows)
  - tasks.json → Build tasks
- **app/** → PoC Application
  - **port/** → Platform-specific implementations
    - port_comm.c → MQTT communication layer
    - port_delay.c →Timing functions
    - port_spi_usb.c → USB/SPI token interface
  - main_rw_cw.c → Main application (interactive menu)
  - Makefile → Build system
- **sdk_fw/** → SGR SDK (read-only)
  - **docs** → Documentation about SDK
  - **examples** → Some examples of PORT layer implementation
  - **include/** → Public SDK headers
  - **port/** → Port layer interface headers
  - **src/** → SDK source code
- setup-debug.sh → Debug configuration script (Linux only)

Key Points:

- The sdk_fw/ directory contains the SDK core and should not be modified
- Configuration is done exclusively through sdk_fw/include/sdk_sgr_config.h
- The app/ directory contains the PoC application and platform-specific code

7.3. Target Environment

The PoC has been developed and tested in the following environment:

- Operating System: Linux (Ubuntu)
- Execution mode: User-space application
- Communication: MQTT over TCP/IP
- Build system: Make

Porting or executing the PoC on a different environment may require adaptations outside the scope of this document.

7.4. PoC Dependencies

This section lists the tools and libraries required only for building and running the PoC. These dependencies are not required to use the SDK on other platforms (e.g., embedded MCUs), where equivalent platform-specific implementations are expected.

7.4.1. GCC and Build Tools

The PoC is written in C and requires a standard GCC toolchain.

Install the required build tools using:

sudo apt update

sudo apt install build-essential

This installs:

- GCC compiler
- Make build system
- Standard C libraries and headers

7.4.2. MQTT Dependencies (Mosquitto)

The PoC uses MQTT for server communication.

The Mosquitto client development library is required:

sudo apt install libmosquitto-dev

This library provides:

- MQTT client headers
- MQTT client library used during linking

7.5. Building the PoC

Step 1: Navigate to the app/ directory and execute:

cd app

make

The build process:

1. Compiles all SDK source files from **../sdk_fw/src/**
2. Compiles the PoC application and port implementations
3. Links everything into the `test_sdk` executable

If the build completes successfully, the executable `test_sdk` will be generated in the `app/` directory.

Step 2: Clean Build

make clean

make

7.6. Running the PoC

After a successful build, execute the PoC application from the SDK root directory:

sudo ./test_sdk

The application presents an interactive menu that allows you to test both normal operation and simulated fault scenarios:

```
============================================================
                  SGR SDK DEMO MENU
============================================================
 1) Normal flow (no faults)
 2) COMM TX error
 3) COMM RX error
 4) SPI TX error
 5) SPI RX error
 0) Exit
------------------------------------------------------------
 Select option:
```

Each option exercises the complete mutual authentication flow between the token and the server, demonstrating how the SDK handles different failure conditions through its built-in retry mechanism.

7.6.1. Option 1: Normal Flow (No Faults)

This option executes the complete authentication sequence without any injected faults. The process consists of two main phases.

- **Phase 1: Token → Server Authentication**

Step 1: Requesting Challenge Word from Server

The SDK reads the Token ID (TID) from the token via SPI, then sends it to the server via MQTT to request a Challenge Word (CW):

```
[MW] ▶ Getting Device ID...
[MW] ✓ Device ID read successfully
[MW] ID (HEX):
800000000000000000000000000000185dcba84764025caa98c6c43032c8
b433
[MW] TID CRC: ce78
[PORT][MQTT] Connected
[PORT][MQTT] Publish OK
[PORT][MQTT] RX received (228 bytes)
[MW] ✓ CW received successfully
[MW] Transaction ID: c1c447bb-da51-4fd4-b1ec-382be4172f4d
```

Step 2: Computing and Replying Response Word to Server

The SDK sends the CW to the token via SPI, which computes the Response Word (RW) using its internal secret key:

```
[MW] ▶ Computing RW from CW
[MW] ✓ RW computed successfully
[MW] CW  (HEX):
9353406d98a39665ec9e5d3ed4893e1e75f0a131a5101cc362efe5afbf8a
0dee
[MW] RW  (HEX): 2d9e58ef7b47f319e7601fa065b94c2c
[PORT][MQTT] Publish OK
[MW] ✓ Authentication SUCCESS
```

The server responds with "auth_status":"AUTH_OK", completing the first phase.

- **Phase 2: Server → Token Authentication**

Step 1: Generating Challenge Word from Token

The SDK generates a new CW locally by duplicating the RW computed in the previous phase:

```
[MW] ▶ Generating CW with TOken
CW generated
(HEX):2D9E58EF7B47F319E7601FA065B94C2C2D9E58EF7B47F319E7601FA065B94C2C
[MW] ✓ CW successfully generated from RW
```

Step 2: Verifying Response Word (Server vs Token)

The SDK sends this CW to the server and requests the server's computed RW. Simultaneously, it sends the same CW to the local token to compute its own RW:

```
[MW] ▶ Requesting RW from server
[PORT][MQTT] Publish OK
[MW] ▶ Computing RW on token (chip)
RW computed on token:0E579EF826AEC610CB5CFA25B81DDB69
RW received from server:0E579EF826AEC610CB5CFA25B81DDB69
[MW] ✓ RW match (AUTH OK)
```

Both RWs match, confirming the server also possesses the correct secret key.

Step 3: Reporting Authentication Result to Cloud

```
[MW] ▶ TX → Cloud (182 bytes)
[PORT][MQTT] Publish OK
[MW] ✓ Authentication Result successfully sent to Cloud
```

Final Result:

```
[MAIN] TOKEN → SERVER AUTH OK
[MAIN] SERVER → TOKEN AUTH OK
[MAIN] DEMO FLOW COMPLETED SUCCESSFULLY
```

7.6.2. Option 2: COMM TX Error

This option injects a fault into the MQTT transmission path by corrupting byte 26 of the outgoing JSON payload. The fault is injected only during the first 3 attempts of Step 1.

Expected Behavior

- **Attempts 1-3: Fault Injected**

The corrupted TID is transmitted to the server, which detects a CRC mismatch:

```
[MW] ID (HEX): 800000000000000000000000000000185dcba84764025caa98c6c43032c8b433
[FAULT]    Injection in byte 26 Tx .... json tx =
{"type":"0","payload":{"TID":"800000000000000000000F000000000185dcba84764025caa98c6c4
3032c8b433"}...


[PORT][MQTT] Publish OK
[PORT][MQTT] RX received (151 bytes) |
data={"type":"0","status":{"code":12,"result":"CRC_INVALID"},"payload":{"reason":"CRC_MISMAT
CH","key":"TID","expected":"ce78","computed":"0xF5ED"}...
[MW] Rx Error (code = 12)
```

The SDK automatically retries. Attempts 2 and 3 fail with the same error.

- **Attempt 4: Fault Cleared**

On the 4th attempt, the fault injection is disabled and the transmission succeeds:

```
[APP] Attempt 4/5

[MW] ID (HEX): 800000000000000000000000000000185dcba84764025caa98c6c43032c8b433

[PORT][MQTT] Publish OK

[PORT][MQTT] RX received (228 bytes) | data={"type":"0","status":{"code":0,"result":"OK"}...

[MW] ✓ CW received successfully
```

The authentication flow then completes normally, demonstrating the SDK's resilience to transient communication errors.

- **Final Result:**

```
[MAIN] TOKEN → SERVER AUTH OK

[MAIN] SERVER → TOKEN AUTH OK

[MAIN] DEMO FLOW COMPLETED SUCCESSFULLY
```

7.6.3. Option 3: COMM RX Error

This option injects a fault into the MQTT reception path by corrupting byte 70 of the incoming JSON payload. The fault is persistent across all retry attempts.

Expected Behavior

The server sends a valid CW, but the SDK receives a corrupted version:

```
[APP] Attempt 1/5

[PORT][MQTT] RX received (228 bytes) |
data={"type":"0","status":{"code":0,"result":"OK"},"payload":{"CW":"4ff554e2a5b9a2ac15604f45
52e35cd4c6959bd8b908855041fa3203e93bab54"...

[FAULT]    [RX] Payload corrupted at byte 70

[MW] ◀ RX ← Server (228 bytes)

{"type":"0","status":{"code":0,"result":"OK"},"payload":{"CW":"4ff554eta5b9a2ac15604f4552e35
cd4c6959bd8b908855041fa3203e93bab54"...

[MW] ✗ CW CRC invalid
```

The SDK retries up to the maximum configured attempts (5), but since the fault is persistent, all attempts fail. The output shows:

```
[APP] Attempt 1/5
[MW] ✗ CW CRC invalid
[APP] Attempt 2/5
[MW] ✗ CW CRC invalid
[APP] Attempt 3/5
[MW] ✗ CW CRC invalid
[APP] Attempt 4/5
[MW] ✗ CW CRC invalid
[APP] Attempt 5/5
[MW] ✗ CW CRC invalid
```

- **Final Result:**

```
[MAIN] TOKEN → SERVER AUTH FAILED (status=9)
```

This demonstrates that the SDK correctly detects corrupted data and exhausts its retry mechanism before reporting failure.

7.6.4. Option 4: SPI TX Error

This option injects a fault into the SPI transmission path by corrupting byte 20 of the outgoing buffer to the token. The fault is persistent across all retry attempts.

Expected Behavior

- **First SPI Call (crypto_get_id): No Impact**

When the SDK reads the Token ID, the fault corrupts byte 20 of the 37-byte SPI TX buffer. However, the critical data (command header) is located in bytes 0-4, so the corruption falls into the padding area and does not affect the operation:

```
[MW] ▶ Getting Device ID...
[FAULT][SPI][TX] Byte 20 corrupted
[MW] ✓ Device ID read successfully
[MW] ID (HEX): 80000000000000000000000000000185dcba84764025caa98c6c43032c8b433
```

- **Second SPI Call (crypto_get_rw): Critical Impact**

When the SDK sends the 64-byte Challenge Word to the token, byte 20 now falls within the CW payload (bytes 5-69 of the 87-byte buffer). The token receives a corrupted CW and computes an incorrect RW:

When the SDK sends the 32-byte Challenge Word to the token via SPI, byte 20 now falls within the CW payload (bytes 5-37 of the 87-byte SPI buffer). The token receives a corrupted CW and computes an incorrect RW. Note that the CW is 32 bytes raw in SPI, but is represented as 64 hex characters when transmitted over MQTT:

```
[APP] Attempt 1/5
[MW] ▶ Computing RW from CW
[FAULT][SPI][TX] Byte 20 corrupted
            ^^^^^^^^^^^^^^^^^^
      CRITICAL: Byte 20 is inside the 64-byte CW being sent to the token
[MW] ✓ RW computed successfully
[MW] CW  (HEX): 93f1427659c979e21f84e969e38eb4f548293eea72596078e9d3515d25ad6bf7
[MW] RW  (HEX): 2da0801f7ef9a55eef70ce0e8e460baf
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
      Token computed RW from CORRUPTED CW → RW is incorrect
```

The SDK sends this incorrect RW to the server:

```
[PORT][MQTT] Publish OK
[PORT][MQTT] RX received (95 bytes) |
data={"type":"1","status":{"code":0,"result":"OK"},"payload":{"auth_status":"AUTH_NOT_OK"}...
[MW] ✗ Authentication FAILED
```

Since the fault is persistent, all retry attempts (1-5) fail with AUTH_NOT_OK.

- **Final Result:**

```
[MAIN] TOKEN → SERVER AUTH FAILED (status=9)
```

This scenario demonstrates the SDK's ability to detect authentication failures caused by data corruption in the SPI communication layer.

7.6.5. Option 5: SPI RX Error

This option injects a fault into the SPI reception path by corrupting byte 20 of the incoming buffer from the token. The fault is persistent across all retry attempts.

Expected Behavior

When the SDK reads the Token ID via SPI, byte 20 of the 37-byte RX buffer is corrupted. Since the Token ID starts at byte 5 and extends for 32 bytes, byte 20 falls within the ID payload:

```
[APP] Attempt 1/5
[MW] ▶ Getting Device ID...
[FAULT][SPI][RX] Byte 20 corrupted
[MW] ✓ Device ID read successfully
[MW] ID (HEX): 800000000000000000000000000000ff5dcba84764025caa98c6c43032c8b433
                                                ^^
                                    CORRUPTION: '18' became 'ff'
```

The SDK sends this corrupted TID to the server:

```
[PORT][MQTT] TX about to write |
topic=sandgrain/v1/800000000000000000000000000000185dcba84764025caa98c6c43032c8
b433/req/requestCW

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
NOTE: Topic uses UNCORRUPTED TID (read at init before fault was active)

data={"type":"0","payload":{"TID":"800000000000000000000000000000ff5dcba84764025caa98
c6c43032c8b433"}...
                                                                    ^^
                                    But JSON payload contains CORRUPTED TID
[PORT][MQTT] Publish OK
[PORT][MQTT] RX received (100 bytes) |
data={"type":"0","status":{"code":10,"result":"NOT_OK"},"payload":{"error":"MissingTokenError"}.
..
[MW] Rx Error (code = 10)
```

The server receives a TID that does not exist in its database and responds with MissingTokenError. Since the fault is persistent, all retry attempts (1-5) fail with the same error.

Final Result:

```
[MAIN] TOKEN → SERVER AUTH FAILED (status=9)
```

This scenario demonstrates that corrupted token identification data is detected by the server's database lookup mechanism.

7.7. Debugging the PoC

The PoC includes Visual Studio Code debug configurations for both Linux and Windows.

7.7.1. Debug Setup (Linux Only)

Linux debugging requires elevated permissions for USB access. Run the setup script once:

```
cd PoC_SGR
./setup-debug.sh
```

This script:

- Configures passwordless sudo for GDB
- Enables debugging without manual password entry

7.7.2. Starting a Debug Session

- Step 1: Open the project in VSCode, if it is not already open:

  ```
  cd PoC_SGR
  code .
  ```

- Step 2: Press F5 to start debugging
- Step 3: Select the appropriate configuration:

  (Linux) Debug PoC

  (Windows) Debug PoC

7.7.3. Setting Breakpoints

- Click on the left margin of any line in the source code to set a breakpoint (red dot appears).
- The debugger will pause execution when it reaches that line.
- Important: Avoid setting breakpoints directly on printf() statements, as GDB may not step through them correctly. Instead, set breakpoints on the line immediately after the printf().

7.8. MQTT Broker Configuration

An MQTT broker must be running before executing the PoC.

The PoC has been validated using an MQTT broker with a web-based monitoring dashboard (e.g., EMQX).

The broker is used to:

- Accept client connections from the PoC
- Exchange authentication-related messages
- Allow inspection of message flow during execution

Ensure that:

- The broker is running before starting the PoC
- Broker connection parameters match the SDK configuration
- Required topics are available and accessible

7.9. Notes and Limitations

- All PoC-specific dependencies are limited to the Linux-based reference implementation.
- The PoC relies on the Portability Layer to access platform services.
- No changes to the Application, Middleware, or Services layers are required to run the PoC.
- The PoC should be considered a reference and validation tool, not a production application.